

Entertainment technology

bachelor in de elektronica – ICT – Brugge

docent: Van Gaever T.

academiejaar 2026-2027

Opdracht 3: ADC Potentiometer – MIDI Control

Change



1. Inleiding

In deze opdracht breid je jouw MIDI controller uit met analoge ingangen. Je sluit potentiometers aan op de ADC (Analog-to-Digital Converter) van de STM32 en gebruikt DMA (Direct Memory Access) om de waarden continu en zonder CPU-belasting in te lezen. De ADC-waarden worden omgezet naar MIDI Control Change (CC) berichten en via USB verstuurd.

Het project is opgedeeld in twee fasen: eerst maak je de volledige keten werkend met één potentiometer, daarna breid je uit naar meerdere kanalen.

2. MIDI control change berichten

MIDI Control Change (CC) berichten worden gebruikt om continue parameters te sturen, zoals volume, pan, of effecten. Een CC bericht bestaat uit drie bytes:

Byte	Inhoud	Bereik
Status byte	0xB0 + kanaal	0xB0 – 0xBF (kanaal 1–16)
Data byte 1	Controller nummer (CC#)	0 – 127
Data byte 2	Waarde	0 – 127

Voorbeeld: een potentiometer stuurt CC#16 op kanaal 1 met de waarde 64 → bytes: 0xB0, 0x10, 0x40.

3. Fase 1: Eén potentiometer

Doel: Sluit één potentiometer aan op een ADC-ingang van de STM32. Configureer de ADC met DMA in circulaire modus, getriggerd door een timer op 1 kHz. De ingelezen waarde wordt omgezet naar een 7-bit MIDI CC waarde en via USB MIDI verstuurd.

3.1. Hardware bedrading

Denk na over hoe je de potentiometer op de analoge input pin moet aansluiten om de potentiometer uit te lezen van 0 – 3,3 volt.

Zorg dat je geen kortsluiting maakt met je voeding.

Belangrijk: Gebruik 3.3V als referentiespanning, niet 5V! De ADC van de STM32 is niet 5V-tolerant.

3.1. CubeMX configuratie: TIM6 (ADC Trigger)

De ADC wordt niet continu uitgevoerd, maar getriggerd door een timer. Dit garandeert een vaste samplefrequentie van 1 kHz (elke milliseconde één conversie). We gebruiken TIM6 als trigger omdat dit een eenvoudige "basic timer" is zonder PWM-functionaliteit – ideaal als interne trigger.

Onfiguureer de kloksnelheid van de microcontroller op 250MHz

Parameter	Waarde	Uitleg
Clock Source	Internal Clock	De timer draait op de interne klok (instellen op 250 MHz)
Prescaler (PSC)	249	Deelt de klok door 250: $250 \text{ MHz} / 250 = 1 \text{ MHz}$. Elke timertick duurt nu $1 \mu\text{s}$.
Counter Period (ARR)	999	De timer telt van 0 tot 999 (= 1000 ticks). Bij 1 MHz geeft dit: $1000 \times 1 \mu\text{s} = 1 \text{ ms} \rightarrow 1 \text{ kHz}$ trigger.
Trigger Output (TRGO)	Update Event	Bij elke overflow (counter bereikt ARR) genereert de timer een TRGO-event. Dit event triggert de ADC om een conversie te starten.

3.2. CubeMX configuratie: ADC1 (Fase 1 – 1 kanaal)

De ADC wordt geconfigureerd om één analoog kanaal in te lezen met 8-bit resolutie. We kiezen bewust voor 8 bits omdat MIDI CC slechts 7-bit waarden (0–127) gebruikt – hogere resolutie is dus overbodig en vereenvoudigt de conversie.

Parameter	Waarde	Uitleg
Resolution	8 bits	Levert waarden 0–255. MIDI CC is 7-bit (0–127), dus 8-bit is voldoende. De conversie naar MIDI is eenvoudig: $\text{waarde} \gg 1$ (delen door 2).
Scan Conversion Mode	Disable	Er is slechts één kanaal actief, dus scannen over meerdere kanalen is niet nodig. Bij meerdere kanalen (Fase 2) moet dit aangezet worden.
Continuous Conversion	Disable	De ADC wacht op een externe trigger (TIM6) in plaats van continu te converteren. Dit garandeert een exacte en voorspelbare samplefrequentie.
DMA Continuous Requests	Enable	Zorgt ervoor dat de DMA na elke transfer klaar staat voor de volgende. Zonder deze instelling stopt de DMA na één conversie.
External Trigger Source	Timer 6 TRGO event	Koppelt de ADC aan de TRGO-output van TIM6. Elke keer dat TIM6 overloopt, start de ADC automatisch een nieuwe conversie.
External Trigger Edge	Rising Edge	De ADC triggert op de stijgende flank van het TRGO-sigitaal.

Number of Conversions	1	Er wordt één kanaal per trigger-event geconverteerd.
Rank 1 – Channel	Kies het kanaal dat bij jouw gekozen pin hoort	De rank bepaalt de volgorde van conversie. Met één kanaal is er slechts één rank.
Rank 1 – Sampling Time	24.5 cycles	Langere sample-tijd geeft een stabielere meting. Bij 1 kHz sample rate is er ruim voldoende tijd beschikbaar.

3.3. CubeMX configuratie: GPDMA (Fase 1)

DMA (Direct Memory Access) transporteert de ADC-resultaten automatisch naar het geheugen zonder tussenkomst van de CPU. De processor is hierdoor vrij om andere taken uit te voeren (zoals MIDI berichten versturen).

Parameter	Waarde	Uitleg
Channel	GPDMA1 Channel 0	Selecteer een vrij DMA-kanaal. Channel 0 is standaard beschikbaar.
Request	ADC1	Koppelt dit DMA-kanaal aan de ADC1 peripheral. De DMA weet nu dat data van de ADC komt.
Direction	Peripheral to Memory	Data stroomt van de ADC (peripheral) naar een variabele in RAM (memory).
Mode	Circular	Na elke transfer begint de DMA opnieuw van voren. Zo wordt de buffer continu overschreven met de nieuwste waarde.
Source Data Width	Byte	De ADC is geconfigureerd op 8 bits, dus één byte per conversie.
Destination Data Width	Byte	De bestemmingsvariabele is ook 8 bits breed.
Source Address Increment	Disabled	Het bronadres (ADC data register) is altijd hetzelfde – geen increment nodig.
Dest Address Increment	Disabled	Bij één kanaal schrijven we steeds naar dezelfde variabele. Bij meerdere kanalen (Fase 2) moet dit op Enable staan!

3.4. Code – Fase 1

Onderstaande code toont de kernfuncties voor Fase 1. De variabele `adc_value` wordt **automatisch door DMA gevuld** met de laatste ADC-waarde.

```
#define HYSTERESIS      2
#define MIDI_CC_POT1    16

// Enkele ADC waarde - wordt gevuld door DMA
volatile uint8_t adc_value;
uint8_t last_midi_value = 0;

void ADC_Start(void) {
    HAL_TIM_Base_Start(&htim6);
    HAL_ADC_Start_DMA(&hadc1, (uint32_t*)&adc_value, 1);
}

void process_potentiometer(void) {
    uint8_t new_value = adc_value >> 1; // 8-bit naar 7-bit

    int16_t diff = (int16_t)new_value - (int16_t)last_midi_value;
    if (diff < 0) diff = -diff;

    if (diff >= HYSTERESIS) {
        if (tud_midi_mounted()) {
            uint8_t msg[3] = { 0xB0, MIDI_CC_POT1, new_value };
            tud_midi_stream_write(0, msg, 3);
        }
        last_midi_value = new_value;
    }
}
```

Uitleg bij de code:

- **HYSTERESIS:** voorkomt dat kleine schommelingen (ruis) ongewenste MIDI berichten genereren. Pas alleen de waarde aan als het verschil groter is dan de drempel.
- `adc_value >> 1:` verschuift de 8-bit waarde (0–255) naar 7-bit (0–127) voor MIDI.
- `volatile:` vertelt de compiler dat deze variabele door hardware (DMA) gewijzigd kan worden en niet weg-geoptimaliseerd mag worden.
- `tud_midi_mounted():` controleert of het USB MIDI device verbonden is met de computer voordat we berichten versturen.
- `ADC_Start():` start eerst de timer (die de trigger levert), daarna de ADC met DMA. De volgorde is belangrijk!

3.5. Testprocedure Fase 1

- ☐ Sluit de potentiometer aan: wiper naar ADC-pin, buitenste pinnen naar 3.3V en GND
- ☐ Flash de firmware met de Fase 1 code
- ☐ Open een MIDI monitor (MIDI-view, MIDI-OX of een DAW)
- ☐ Draai aan de potentiometer en controleer of CC 16 berichten verschijnen

- ☐ Controleer het bereik: 0 (helemaal links) tot 127 (helemaal rechts)
- ☐ Controleer stabiliteit: geen jitter wanneer de potentiometer stilstaat

4. Fase 2: Meerdere potentiometers

Doel: Breid de configuratie uit van één naar meerdere potentiometers (bv. 8). Elke potentiometer stuurt een eigen MIDI CC nummer.

Nu je de volledige keten met één potentiometer werkend hebt, breid je uit naar meerdere kanalen. Sluit extra potentiometers aan op verschillende ADC-ingangen. Kies zelf de pinnen – raadpleeg de datasheet en het pinout diagram van de Nucleo-H533RE om geschikte analoge pinnen te vinden die niet in conflict zijn met andere peripherals (SPI, USB).

4.1. CubeMX wijzigingen voor Fase 2

Er zijn slechts twee instellingen in CubeMX die aangepast moeten worden ten opzichte van Fase 1:

ADC1 wijzigingen:

Parameter	Fase 1	Fase 2	Waarom?
Scan Conversion Mode	Disabled	Enabled	De ADC moet nu meerdere kanalen na elkaar scannen in één trigger-cyclus.
Number of Conversions	1	Aantal potentiometers	Elk kanaal heeft een eigen Rank nodig. Voeg voor elke potentiometer een Rank toe met het juiste kanaal.

GPDMA wijziging:

Parameter	Fase 1	Fase 2	Waarom?
Dest Address Increment	Disabled	Enabled	De DMA moet nu opeenvolgende waarden in een array schrijven. Met increment schuift het bestemmingsadres na elke byte één positie op.

4.2. Code – Fase 2

De code wordt aangepast om een array van ADC-waarden te verwerken in plaats van een enkele variabele:

```
#define NUM_POTS      8    // Pas aan naar jouw aantal
#define MIDI_CC_START 16    // Eerste CC nummer
#define HYSTERESIS    2
```

```

volatile uint8_t adc_buffer[NUM_POTS]; // Array i.p.v. enkele waarde
uint8_t last_midi_value[NUM_POTS] = {0};

void ADC_Start(void) {
    HAL_TIM_Base_Start(&htim6);
    HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, NUM_POTS);
}

void process_potentiometers(void) {
    for (int i = 0; i < NUM_POTS; i++) {
        uint8_t new_value = adc_buffer[i] >> 1;
        int16_t diff = (int16_t)new_value - (int16_t)last_midi_value[i];
        if (diff < 0) diff = -diff;

        if (diff >= HYSTERESIS) {
            if (tud_midi_mounted()) {
                uint8_t msg[3] = { 0xB0, MIDI_CC_START + i, new_value };
                tud_midi_stream_write(0, msg, 3);
            }
            last_midi_value[i] = new_value;
        }
    }
}

```

Belangrijkste verschillen met Fase 1:

- `adc_buffer[NUM_POTS]` in plaats van een enkele variabele – DMA vult de array automatisch met de waarden van alle kanalen in de juiste volgorde (bepaald door de Rank-volgorde in CubeMX).
- `MIDI_CC_START + i` wijst automatisch een oplopend CC nummer toe aan elke potentiometer (CC 16, 17, 18, ...).
- De for-loop verwerkt alle potentiometers met dezelfde hysteresis-logica.

4.3. Wat je zelf moet uitzoeken

- Welke pinnen van de Nucleo-H533RE geschikt zijn als analoge ingang (ADC-kanalen) – raadpleeg de datasheet en het CubeMX pinout overzicht.
- Hoe je conflicten vermijdt met pinnen die al in gebruik zijn voor SPI (MCP23S17) of USB.
- Hoeveel ADC-kanalen beschikbaar zijn op ADC1 en of je eventueel ADC2 nodig hebt voor meer kanalen.
- Welke CC nummers je wilt gebruiken (CC 16–31 zijn vrij beschikbaar als "General Purpose Controllers").

4.4. Testprocedure Fase 2

- ☐ Sluit alle potentiometers aan op de gekozen ADC-pinnen
- ☐ Pas de CubeMX configuratie aan (Scan Mode, Ranks, DMA increment)
- ☐ Flash de firmware met de Fase 2 code

- ☐ Test elke potentiometer individueel in de MIDI monitor
- ☐ Controleer dat elke potentiometer een eigen CC nummer stuurt
- ☐ Controleer dat alle potentiometers stabiel zijn (geen jitter bij stilstand)

4.5. Verslag indienen

- Broncode met commentaar
- Screenshot van MIDI monitor met CC berichten van meerdere potentiometers
- Overzicht van jouw pin-toewijzing (welke pin → welk ADC kanaal → welk CC nummer)
- Demonstratievideo waarin je laat zien dat potentiometers MIDI CC waarden sturen.

5. Extra vragen

1. De shift-operatie `adc_value >> 1` zet 8-bit om naar 7-bit. Maar welke waarde krijg je maximaal? Is dat precies 127? Schrijf de bitoperatie uit voor `adc_value = 255`. Wat krijg je bij `adc_value = 254`. Is dit een probleem?
2. De HYSTERESIS is ingesteld op 2. Stel dat de potentiometer stilstaat maar de ADC-waarde schommelt tussen 100 en 101. Worden er dan MIDI berichten verstuurd? En bij schommeling tussen 100 en 103? Leg uit.
3. In `ADC_Start()` wordt eerst `HAL_TIM_Base_Start` aangeroepen en daarna `HAL_ADC_Start_DMA`. Leg uit waarom de volgorde belangrijk is.
4. De main loop draait sneller dan de ADC samplet (1 kHz). Worden er dubbele berichten verstuurd? Leg uit.